Arin Zingade 0801IT221035

# Assignment 4: MapReduce Programming with Java

**Q1: Word Count** — Count the frequency of each word in a text file. 2.

### Source Code

```
public static class TokenizerMapper
 extends Mapper<Object, Text, Text, IntWritable> {

 private final static IntWritable one = new IntWritable(1);
 private Text word = new Text();

 @Override
 public void map(Object key, Text value, Context context)
 throws IOException, InterruptedException {
 StringTokenizer itr = new StringTokenizer(value.toString());
 while (itr.hasMoreTokens()) {
 word.set(itr.nextToken().toLowerCase());
 context.write(word, one);
 }
 }
}

public static class IntSumReducer
 extends Reducer<Text, IntWritable, Text, IntWritable> {

 private IntWritable result = new IntWritable();

 @Override
 public void reduce(Text key, Iterable<IntWritable> values, Context
context)
 throws IOException, InterruptedException {
 int sum = 0;
 for (IntWritable val : values) {
 sum += val.get();
 }
 result.set(sum);
 context.write(key, result);
 }
}
```

Arin Zingade 0801IT221035

## run_wordcount.sh

```
#!/bin/bash
# ============================================================
# Assignment 4 - Q1: Word Count MapReduce
# ============================================================
# Student Name : Arin Zingade
# Enrollment No : 0801IT221035
# Subject : Big Data
# ============================================================

echo "==========================================================="
echo " Big Data Assignment 4 - Q1: Word Count"
echo " Student : Arin Zingade"
echo " Enroll : 0801IT221035"
echo "==========================================================="
echo ""

# Verify HADOOP_HOME is set
if [ -z "$HADOOP_HOME" ]; then
echo "ERROR: HADOOP_HOME is not set. Please run: source ~/.bashrc"
exit 1
fi

cd "$(dirname "$0")/WordCount"

# Compile
echo "[Step 1] Compiling WordCount.java..."
javac -classpath "$(hadoop classpath)" -d . WordCount.java
if [ $? -ne 0 ]; then echo "Compilation failed!"; exit 1; fi

# Package into JAR
echo "[Step 2] Creating wordcount.jar..."
jar -cvf wordcount.jar *.class

# Upload input to HDFS
echo "[Step 3] Uploading input to HDFS..."
hdfs dfs -rm -r -f /ass4_word_count_input
hdfs dfs -mkdir -p /ass4_word_count_input
hdfs dfs -put input/sample_input.txt /ass4_word_count_input/

# Clean up old output
hdfs dfs -rm -r -f /wc_output
```

Arin Zingade 0801IT221035

```
# Run the MapReduce job
echo "[Step 4] Running WordCount MapReduce job..."
hadoop jar wordcount.jar WordCount /ass4_word_count_input /wc_output

# Display output
echo ""
echo "[Step 5] Word Count Output:"
echo "----------------------------------------"
hdfs dfs -cat /wc_output/part-r-00000

echo ""
echo "========================================================"
echo " Q1: Word Count Complete!"
echo "========================================================"
```

```
==========================================================
  Big Data Assignment 4 - Q1: Word Count
  Student : Arin Zingade
  Enroll  : 0801IT221035
==========================================================

[Step 1] Compiling WordCount.java...
[Step 2] Creating wordcount.jar...
added manifest
adding: WordCount$IntSumReducer.class(in = 1793) (out= 775)(deflated 56%)
adding: WordCount$TokenizerMapper.class(in = 1839) (out= 817)(deflated 55%)
adding: WordCount.class(in = 1885) (out= 1045)(deflated 44%)
[Step 3] Uploading input to HDFS...
Deleted /ass4_word_count_input
Deleted /wc_output
[Step 4] Running WordCount MapReduce job...
========================================
  WordCount MapReduce
  Student : Arin Zingade
  Enroll  : 0801IT221035
========================================
2026-04-23 02:28:49,543 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting
2026-04-23 02:28:50,046 WARN mapreduce.JobResourceUploader: Hadoop command-line opt
plication with ToolRunner to remedy this.
2026-04-23 02:28:50,074 INFO mapreduce.JobResourceUploader: Disabling Erasure Codin
0009
2026-04-23 02:28:50,487 INFO input.FileInputFormat: Total input files to process :
2026-04-23 02:28:50,625 INFO mapreduce.JobSubmitter: number of splits:1
2026-04-23 02:28:50,900 INFO mapreduce.JobSubmitter: Submitting tokens for job: job
```

Arin Zingade 0801IT221035

```
[Step 5] Word Count Output:
----------------------------------------
a       1
amazing 1
an      1
apache  1
big     3
data    4
distributed     1
for     2
framework       1
hadoop  4
hello   1
is      3
large   1
mapreduce       3
model   1
of      1
on      1
open    1
parallel        1
process 1
processing      1
programming     1
runs    1
source  1
systems 2
uses    1
volumes 1

============================================================
  Q1: Word Count Complete!
============================================================
```

Arin Zingade 0801IT221035

---

**Q2: Anagram Finder** — Group words that are anagrams of each other.

### Source Code

```
public static class AnagramMapper
        extends Mapper<Object, Text, Text, Text> {

    @Override
    public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {
        String[] words = value.toString().trim().split("\\s+");
        for (String w : words) {
            String cleaned = w.toLowerCase().replaceAll("[^a-z]", "");
            if (cleaned.isEmpty()) continue;

            char[] charArray = cleaned.toCharArray();
            Arrays.sort(charArray);
            String sortedKey = new String(charArray);

            context.write(new Text(sortedKey), new Text(w));
        }
    }
}

public static class AnagramReducer
        extends Reducer<Text, Text, Text, Text> {

    @Override
    public void reduce(Text key, Iterable<Text> values, Context
context)
            throws IOException, InterruptedException {
        StringBuilder sb = new StringBuilder();
        for (Text val : values) {
            if (sb.length() > 0) sb.append(", ");
            sb.append(val.toString());
        }
        // Only output actual anagram groups (more than one word)
        if (sb.toString().contains(",")) {
            context.write(key, new Text(sb.toString()));
        }
    }
}
```

Arin Zingade 0801IT221035

## run_anagram.sh

```bash
#!/bin/bash

# ============================================================
# Assignment 4 - Q2: Anagram Finder MapReduce
# ============================================================
# Student Name : Arin Zingade
# Enrollment No : 0801IT221035
# Subject : Big Data
# ============================================================

echo "============================================================"
echo " Big Data Assignment 4 - Q2: Anagram Finder"
echo " Student : Arin Zingade"
echo " Enroll : 0801IT221035"
echo "============================================================"
echo ""

# Verify HADOOP_HOME is set
if [ -z "$HADOOP_HOME" ]; then
echo "ERROR: HADOOP_HOME is not set. Please run: source ~/.bashrc"
exit 1
fi

cd "$(dirname "$0")/Anagram"

# Compile
echo "[Step 1] Compiling Anagram.java..."
javac -classpath "$(hadoop classpath)" -d . Anagram.java
if [ $? -ne 0 ]; then echo "Compilation failed!"; exit 1; fi

# Package into JAR
echo "[Step 2] Creating anagram.jar..."
jar -cvf anagram.jar *.class

# Upload input to HDFS
echo "[Step 3] Uploading input to HDFS..."
hdfs dfs -rm -r -f /anagram_input
hdfs dfs -mkdir -p /anagram_input
hdfs dfs -put input/sample_input.txt /anagram_input/
```

Arin Zingade 0801IT221035

```
# Clean up old output
hdfs dfs -rm -r -f /anagram_output

# Run the MapReduce job
echo "[Step 4] Running Anagram Finder MapReduce job..."
hadoop jar anagram.jar Anagram /anagram_input /anagram_output

# Display output
echo ""
echo "[Step 5] Anagram Groups Found:"
echo "----------------------------------------"
hdfs dfs -cat /anagram_output/part-r-00000

echo ""
echo "==========================================================="
echo " Q2: Anagram Finder Complete!"
echo "==========================================================="
```

```
=========================================================
  Big Data Assignment 4 - Q2: Anagram Finder
  Student : Arin Zingade
  Enroll  : 0801IT221035
=========================================================

[Step 1] Compiling Anagram.java...
[Step 2] Creating anagram.jar...
added manifest
adding: Anagram$AnagramMapper.class(in = 1948) (out= 911)(deflated 53%)
adding: Anagram$AnagramReducer.class(in = 1942) (out= 874)(deflated 54%)
adding: Anagram.class(in = 1805) (out= 1001)(deflated 44%)
[Step 3] Uploading input to HDFS...
Deleted /anagram_input
Deleted /anagram_output
[Step 4] Running Anagram Finder MapReduce job...
========================================
  Anagram Finder MapReduce
  Student : Arin Zingade
  Enroll  : 0801IT221035
========================================
2026-04-23 02:29:46,381 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting
2026-04-23 02:29:46,715 WARN mapreduce.JobResourceUploader: Hadoop command-line opt
```

Arin Zingade 0801IT221035

```
[Step 5] Anagram Groups Found:
----------------------------------------
act     act, tac, cat
aet     tea, ate, eat
art     tar, art, rat
dgo     dog, god
eilms   limes, slime, smile
eilnst  enlist, silent, listen
eilv    vile, evil

============================================================
  Q2: Anagram Finder Complete!
============================================================
```